Week 22
Constrained MDPs, Lagrangian Methods, CPO, FOCOPS, P3O

STUDY-RL — a self-paced Deep RL curriculum

October 6, 2026

Contents

Learning objectives 3

1 Motivation & intuition 4
- 1.1 The hard problem nobody told you about . . . 4
- 1.2 Three mental models . . . 4

2 Mathematical formulation 5
- 2.1 Altman's CMDP . . . 5
- 2.2 The Lagrangian and its saddle-point . . . 5
- 2.3 Deriving the primal-dual update . . . 6
- 2.4 The CPO trust-region update . . . 7
- 2.5 FOCOPS — first-order non-parametric optimum . . . 7
- 2.6 P3O — penalised PPO . . . 8
- 2.7 Risk-sensitive primer: CVaR and distributional critics . . . 8

3 Algorithms 9
- 3.1 PPO-Lagrangian pseudocode . . . 9
- 3.2 A 5-step worked numerical example . . . 9
- 3.3 CPO pseudocode . . . 10
- 3.4 FOCOPS pseudocode . . . 10
- 3.5 P3O pseudocode . . . 10

4 Implementation walkthrough 11
- 4.1 Environment: a constrained lava-grid . . . 11
- 4.2 Networks . . . 11
- 4.3 GAE on two streams . . . 11
- 4.4 The combined PPO loss . . . 12
- 4.5 Dual update . . . 12

5 Empirical results 13
- 5.1 Setup . . . 13
- 5.2 Results . . . 13
- 5.3 Reading the curves . . . 14

6 Exercises 16

7 Further reading 17

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this lecture you will be able to:

1. State Altman's Constrained MDP (CMDP) formulation as both a constrained optimisation in policy space and as a linear program in occupancy-measure space.
2. Derive the Lagrangian relaxation step-by-step, identify the saddle-point and write the primal-dual algorithm.
3. Implement PPO-Lagrangian end-to-end: dual advantage, projected multiplier update by projected descent on L, gradient-clipping, and the variance-control tricks (advantage normalisation, cost-scale normalisation).
4. Derive the CPO trust-region update from first principles, including the closed-form solution of the constrained quadratic program.
5. State and apply the FOCOPS closed-form non-parametric optimum and the P3O exact-penalty argument.
6. Define a coherent risk measure, derive Conditional Value-at-Risk, and read off CVaR from a distributional critic.
7. Diagnose the two canonical failure modes of Lagrangian RL (multiplier oscillation, cost-scale mismatch) and explain why PID-Lagrangian damps them.

1 Motivation & intuition

1.1 The hard problem nobody told you about

Up to week 21 we have been maximising expected return $J_r(\pi) = \mathbb{E}_\pi[\sum_t \gamma^t r_t]$. In practice this is rarely what we actually want.[1] The deployer almost always also cares about a side-quantity that must not blow up: an energy budget, a collision count, an unfairness metric, a maximum exposure to a particular candidate group, a clinical safety event rate.

In the unconstrained formulation those concerns get baked into the reward through a hand-tuned scalar: $r_t - \beta\, c_t$. This is brittle. The chosen β is almost always either too soft (constraint violated) or too harsh (reward collapses). Worse, the trade-off interpretation of β is *not* preserved during training — the agent has no view of the global constraint, only of the local linear combination.

The Constrained MDP framework promotes that scalar to a first-class variable $\lambda \geq 0$ that the algorithm *learns*, by formal duality. The result is a primal-dual algorithm that[2] solves *the right problem* — maximise reward subject to constraint.

1.2 Three mental models

Intuition. **The thermostat analogy.** λ is the price of cost. When the agent is overspending the budget, prices rise and the policy starts avoiding cost; when it is underspending, prices fall and the policy can afford to chase reward more aggressively. The equilibrium price is the marginal value of one extra unit of budget.

Intuition. **The market analogy.** Each (state, action) pair has two coupons: $r(s, a)$ and $c(s, a)$. The dual variable λ is the exchange rate between them. The optimal policy trades reward for cost at exactly that exchange rate; at any other rate, the budget is either wasted (slack constraint) or violated.

Intuition. **The KKT analogy.** At the saddle point of the Lagrangian we have stationarity (policy gradient zero), primal feasibility ($J_c \leq d$), dual feasibility ($\lambda \geq 0$), and complementary slackness ($\lambda(J_c - d) = 0$). Same KKT conditions you saw in convex analysis — but evaluated through Monte-Carlo estimates on policy-induced trajectory distributions.

[1] In recruiting, a "return" might be "successful hires" and a "cost" might be "rate of demographic-parity violation". Maximising one with zero regard for the other is an excellent way to build a discrimination-as-a-service product.

[2] This is exactly the same trick that turns Support-Vector-Machine training into the dual problem you saw in your ML course: a constrained quadratic program with KKT conditions, solved by alternating primal–dual updates.

2 Mathematical formulation

2.1 Altman's CMDP

Definition 2.1.1 (Constrained MDP)**.** A *constrained Markov Decision Process* is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, c, \gamma, d)$ where $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ is a standard discounted MDP and $c : \mathcal{S} \times \mathcal{A} \to \mathbb{R}_{\geq 0}$ is a *cost function* with budget $d \geq 0$. The objective is

$$\max_{\pi} \; J_r(\pi) \triangleq \mathbb{E}_\pi\left[\sum_{t\geq 0} \gamma^t r(s_t, a_t)\right] \quad \text{subject to} \quad J_c(\pi) \triangleq \mathbb{E}_\pi\left[\sum_{t\geq 0} \gamma^t c(s_t, a_t)\right] \leq d.$$

In full generality there can be several cost channels with separate budgets; we keep one for clarity.

The LP view. For a finite CMDP, the *discounted state-action occupancy measure*

$$\mu^\pi(s, a) \;=\; (1-\gamma)\sum_{t\geq 0} \gamma^t \mathbb{P}_\pi(s_t = s, a_t = a)$$

satisfies $\sum_{s,a} \mu^\pi(s, a) = 1$ and the Bellman-flow constraint

$$\sum_a \mu(s, a) \;=\; (1-\gamma)\rho_0(s) + \gamma \sum_{s',a'} P(s \mid s', a')\, \mu(s', a').$$

The CMDP becomes a linear program over the polytope of valid occupancies:

$$\max_{\mu} \; \tfrac{1}{1-\gamma}\sum_{s,a} \mu(s, a)\, r(s, a) \;\text{ s.t. }\; \tfrac{1}{1-\gamma}\sum_{s,a} \mu(s, a)\, c(s, a) \leq d, \;\; \text{Bellman flow.}$$

Because there are m scalar constraints, an optimal vertex of this polytope needs at most m randomisations beyond the basic deterministic feasible set — this is Altman, 1999's classical structural theorem. In practice it means the *optimal policy in a CMDP can be stochastic* even when the unconstrained optimum was deterministic — a fact you must internalise before you start debugging your safety algorithms.

Common pitfall

Determinism bias. Many practitioners default to deterministic policies because TD3/DDPG/SAC-tuned habits push that way. In a CMDP a constraint *may require randomisation at the optimal tight solution.* A deterministic policy can be feasible or even optimal, but restricting to deterministic policies can lose value or feasibility in some CMDPs.

2.2 The Lagrangian and its saddle-point

Form the (one-constraint) Lagrangian

$$L(\pi, \lambda) \;=\; J_r(\pi) - \lambda\,(J_c(\pi) - d), \qquad \lambda \geq 0. \tag{2.1}$$

Theorem 2.2.1 (Strong duality for finite CMDPs)**.** *If the CMDP from definition 2.1.1 is feasible (i.e., there exists π_0 with $J_c(\pi_0) < d$, Slater's condition), then strong duality holds:*

$$\max_{\pi} \min_{\lambda \geq 0} L(\pi, \lambda) \;=\; \min_{\lambda \geq 0} \max_{\pi} L(\pi, \lambda).$$

A primal-dual optimal pair $(\pi^\star, \lambda^\star)$ exists and the KKT conditions are

$$\begin{aligned} \textit{primal feas.} \quad & J_c(\pi^\star) \leq d, \\ \textit{dual feas.} \quad & \lambda^\star \geq 0, \\ \textit{compl. slack.} \quad & \lambda^\star (J_c(\pi^\star) - d) = 0, \\ \textit{stationarity} \quad & \pi^\star \in \arg\max_{\pi} \; J_r(\pi) - \lambda^\star J_c(\pi). \end{aligned}$$

Proof sketch. Re-write the CMDP as the LP above. The feasible polytope is bounded by Bellman flow + simplex constraints + the linear cost constraint. The dual LP has variables $\lambda \geq 0$ (for the cost constraint) and free variables for the Bellman-flow constraints. Slater $\Rightarrow$ the primal LP is strictly feasible $\Rightarrow$ strong duality for LP (no gap). KKT is the LP optimality criterion. Translating dual variables back to policies: the inner $\max_\pi$ at fixed λ is an unconstrained MDP with reward $r - \lambda c$, hence has a deterministic optimum — and a randomisation between two such MDPs (corresponding to $\lambda \pm \epsilon$) realises the constrained optimum. □

Remark. For deep RL, the inner $\max_\pi$ at fixed λ is approximate (truncated gradient ascent on the surrogate). We do not get exact KKT in finite time; convergence requires additional assumptions on optimisation, estimation, feasibility, and timescales. A Robbins–Monro multiplier step-size schedule alone does not guarantee a saddle point for a neural policy.

2.3 Deriving the primal-dual update

Suppose π_θ is parametrised by θ. We want to ascend L in θ and descend L in λ:

$$\theta_{k+1} = \theta_k + \eta_\theta \, \nabla_\theta L(\pi_{\theta_k}, \lambda_k) = \theta_k + \eta_\theta \, (\nabla_\theta J_r(\pi_{\theta_k}) - \lambda_k \, \nabla_\theta J_c(\pi_{\theta_k})).$$

Both gradients are policy-gradient quantities:

$$\nabla_\theta J_r(\pi_\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \, A_r^{\pi_\theta}(s_t, a_t) \right],$$

and analogously for cost. The combined gradient uses the *augmented advantage*

$$A^{\text{tot}}(s,a) \;=\; A_r(s,a) - \lambda \, A_c(s,a), \tag{2.2}$$

which is the cleanest object to remember from this lecture.

For the multiplier we use projected gradient descent on L at fixed π. Since $\partial L / \partial \lambda = -(J_c - d)$, descent increases λ when cost exceeds the budget:

$$\lambda_{k+1} \;=\; \big[\, \lambda_k + \eta_\lambda \, (\hat{J}_c(\pi_{\theta_k}) - d) \, \big]_+. \tag{2.3}$$

$\hat{J}_c$ is the Monte-Carlo estimate from the current rollout.

Why we don't simply use $r - \beta c$. A fixed β is *exactly* the Lagrangian *at the optimum* $\beta = \lambda^\star$. Of course we do not know $\lambda^\star$ a priori. The primal-dual loop computes it for us.

2.4 The CPO trust-region update

Achiam et al. 2017 (arXiv:1705.10528) gave the first deep-RL algorithm with a per-iteration constraint-satisfaction guarantee. The key inequality, derived from the standard performance-difference lemma (Kakade & Langford 2002):

$$J_c(\pi') - J_c(\pi) \;\le\; \tfrac{1}{1-\gamma}\,\mathbb{E}_{s\sim d^{\pi},a\sim\pi'}\big[A_c^{\pi}(s,a)\big] \;+\; \tfrac{\sqrt{2}\,\gamma\,\epsilon_c^{\pi}}{(1-\gamma)^2}\,\sqrt{\bar{D}_{\mathrm{KL}}(\pi'\,\|\,\pi)}, \tag{2.4}$$

where $\epsilon_c^{\pi} \triangleq \max_s |\mathbb{E}_{a\sim\pi'}[A_c^{\pi}(s,a)]|$ is a worst-case cost-advantage scale. At each iteration CPO solves

$$\begin{aligned} \theta_{k+1} = \arg\max_{\theta}\; & g^{\top}(\theta-\theta_k) \\ \text{s.t.}\; & b^{\top}(\theta-\theta_k) + c \;\le\; 0, \\ & \tfrac{1}{2}(\theta-\theta_k)^{\top} H (\theta-\theta_k) \;\le\; \delta, \end{aligned} \tag{2.5}$$

where $g = \nabla_\theta J_r$, $b = \nabla_\theta J_c$ (cost gradient evaluated under the current policy), $c = J_c(\pi_k) - d$ (current constraint violation), H the Fisher information of π_θ at θ_k, and δ is the trust-region radius.

Theorem 2.4.1 (CPO closed-form update)**.** *Define $q = g^{\top}H^{-1}g$, $r_{bg} = g^{\top}H^{-1}b$, $s_b = b^{\top}H^{-1}b$. Assume H is positive definite and $s_b > 0$. The linearised constraint is feasible inside the trust region iff $c \le \sqrt{2\delta s_b}$. In the regular case with an active trust region, the optimal step is*

$$\theta_{k+1} - \theta_k \;=\; \tfrac{1}{\lambda_\star}\,H^{-1}(g - \nu_\star b),$$

where $\lambda_\star, \nu_\star$ solve the dual

$$\min_{\lambda\ge 0,\,\nu\ge 0}\; \tfrac{1}{2\lambda}(q - 2\nu r_{bg} + \nu^2 s_b) - \nu c + \lambda\delta, \qquad \lambda > 0.$$

For an infeasible linearised constraint ($c > 0$ and the trust region cannot satisfy it), CPO performs the recovery step $\theta_{k+1} = \theta_k - \sqrt{2\delta/s_b}\,H^{-1}b$, *which is the pure cost-reduction step inside the trust region.*

Proof. Form the Lagrangian $\mathcal{L}(\theta,\lambda,\nu) = -g^{\top}(\theta-\theta_k) + \nu(b^{\top}(\theta-\theta_k)+c) + \lambda(\tfrac{1}{2}(\theta-\theta_k)^{\top}H(\theta-\theta_k)-\delta)$. Stationarity in θ gives $(\theta-\theta_k) = H^{-1}(g-\nu b)/\lambda$. Substituting back gives the concave dual of the negative-reward minimisation. Negating that dual yields the displayed minimisation; this changes both the νc and $\lambda\delta$ signs. KKT classifies feasible vs. recovery regimes. □

CPO comes with a per-iteration bound: $J_c(\pi_{k+1}) \le d + (\text{trust-region slack})$, which decays as $\sqrt{\delta}$.

Common pitfall

Fisher inversion. $H^{-1}g$ in CPO is computed by conjugate gradients on $Hv = g$, not by forming H explicitly. Each CG iteration costs one Hessian-vector product (forward + double backward through the log-policy). For 24–32 GB single-GPU budgets, choose CG iterations *at most* 10–20; beyond that you pay more than the policy step itself.

2.5 FOCOPS — first-order non-parametric optimum

Zhang, Vuong, Ross 2020 (arXiv:2002.06506). FOCOPS solves the same problem as CPO but in non-parametric policy space (working point: at fixed π_k, what is the optimal π'?), then projects.

Starting from the Lagrangian of the CPO QP *in non-parametric form*,

$$\max_{\pi'} \mathbb{E}_{s\sim d^{\pi_k}, a\sim\pi'}[A_r^{\pi_k}(s,a)] \text{ s.t. } \mathbb{E}[A_c^{\pi_k}(s,a)] + c \le 0, \;\; D_{\mathrm{KL}}(\pi'\|\pi_k) \le \delta,$$

and forming the Lagrangian $\mathcal{L}(\pi', \nu, \lambda)$, the KKT stationarity in $\pi'(a|s)$ gives

$$\log \pi'(a|s) = \log \pi_k(a|s) + \tfrac{1}{\lambda}\big(A_r^{\pi_k}(s,a) - \nu A_c^{\pi_k}(s,a)\big) - 1 + \mu(s),$$

where $\mu(s)$ is the multiplier on the row-normalisation $\sum_a \pi'(a|s) = 1$. Exponentiating and absorbing $\mu(s)$ into the normaliser:

$$\boxed{\pi^\star(a|s) \;\propto\; \pi_k(a|s)\, \exp\Big(\tfrac{1}{\lambda}\big(A_r^{\pi_k}(s,a) - \nu A_c^{\pi_k}(s,a)\big)\Big).} \tag{2.6}$$

Two free multipliers (λ, ν) are determined by the KL and cost constraints, both 1-D root-finds.

The *projection* step then minimises $D_{\mathrm{KL}}(\pi^\star\|\pi_\theta)$ over θ in the parametric family (a vanilla supervised step), giving the parameter update.

Why FOCOPS is friendlier than CPO on a single GPU. No Fisher-vector products, no conjugate gradients — both phases are first-order. The price: the trust-region inequality eq. (2.4) is respected *only on average over states*, not worst-case per-state.

2.6 P3O — penalised PPO

Zhang, Vuong, Ross 2022 (arXiv:2205.11814). The exact-penalty trick:

$$L_{\mathrm{P3O}}(\theta) \;=\; L_r^{\mathrm{PPO}}(\theta) - \kappa\,[J_c(\pi_\theta) - d]_+. \tag{2.7}$$

Theorem 2.6.1 (Exact penalty)**.** *Under mild Slater-type assumptions on the CMDP, there exists $\kappa^\star < \infty$ such that for any $\kappa \ge \kappa^\star$, the unconstrained minimiser of eq. (2.7) is also a feasible (and in fact optimal) solution of the original constrained problem.*

The hinge $[\cdot]_+$ is non-smooth; in practice we take a subgradient (κ above d, 0 below) and PPO does the rest.

2.7 Risk-sensitive primer: CVaR and distributional critics

Definition 2.7.1 (Conditional Value-at-Risk)**.** For a random variable G with c.d.f. F_G and level $\alpha \in (0, 1]$,

$$\mathrm{VaR}_\alpha(G) = \inf\{g : F_G(g) \ge \alpha\}, \qquad \mathrm{CVaR}_\alpha^{\mathrm{lower}}(G) = \tfrac{1}{\alpha}\int_0^\alpha \mathrm{VaR}_u(G)\, du.$$

This lower-tail return functional captures the worst α fraction of outcomes, fractionally weighting an atom at the quantile when necessary. For continuous distributions it equals the conditional mean below the quantile. The corresponding upper-tail loss risk measure is coherent; do not transfer the sub-additivity sign unchanged to a lower-tail reward utility. Here $\alpha \to 1$ recovers $\mathbb{E}[G]$, $\alpha \to 0$ recovers worst-case.

With a distributional critic $Z^\pi(s,a)$ (C51, QR-DQN, IQN — introduced in Week 6), CVaR becomes a *post-hoc* computation on the empirical quantile function. The risk-sensitive greedy action is

$$a^\star(s) \;=\; \arg\max_a \; \mathrm{CVaR}_\alpha(Z(s,a)).$$

For a cost variable C, the worst α fraction is the *upper* tail, $\alpha^{-1}\int_{1-\alpha}^1 \mathrm{VaR}_u(C)\, du$. Week 24's shipped implementation uses a mean parity statistic; it does not implement a CVaR cost constraint.

Intuition. Expected cost describes the average. Upper-tail CVaR at tail mass 0.05 describes the worst 5% of cost outcomes. Lower-tail CVaR of cost instead selects the smallest costs and would answer the wrong safety question.

3 Algorithms

3.1 PPO-Lagrangian pseudocode

Algorithm: PPO-Lagrangian (one update)

Inputs: policy π_θ, value heads V_r, V_c, multiplier λ, budget d.

1. Collect T steps of π_θ rollout, storing $\{(s_t, a_t, r_t, c_t, V_r(s_t), V_c(s_t), \log \pi_\theta(a_t|s_t))\}$.
2. Compute GAE advantages A_r on reward, A_c on cost (with respective bootstrap targets). *Normalise* A_r to zero mean / unit std. *Do not* normalise A_c — the multiplier needs to see the raw cost scale.
3. Combine: $A^{\text{tot}} = \dfrac{A_r - \lambda A_c}{1+\lambda}$. The $1+\lambda$ denominator keeps surrogate gradient magnitudes bounded as λ grows.
4. PPO update on π_θ with A^{tot}, clip ratio ϵ, K epochs, minibatches of size M. Train V_r, V_c by MSE to GAE returns.
5. Dual update: $\lambda \leftarrow \left[\lambda + \eta_\lambda(\hat{J}_c - d)\right]_+$, clipped to $[0, \lambda_{\max}]$.

Complexity. Each update is $\mathcal{O}(T \cdot K)$ forward passes plus the same many backward passes. The multiplier update is $\mathcal{O}(1)$. Storage is $\mathcal{O}(T)$ for the rollout buffer.

3.2 A 5-step worked numerical example

Consider a 2-state, 2-action MDP with $r = [[0, 0], [0, +1]]$ and $c = [[0, +1], [0, +1]]$ (rows are states, cols are actions: action 1 always costs 1, action 1 in state 2 also pays 1). Set $\gamma = 0.9$, budget $d = 0.5$, and discount the agent's first action.[1]

A randomly initialised policy plays action 1 with probability 0.5 everywhere, giving $\hat{J}_c \approx 0.5/(1-0.9) = 5$ (vastly over budget). One dual step with $\eta_\lambda = 0.1$: $\lambda \leftarrow \max(0, 0 + 0.1(5 - 0.5)) = 0.45$. On the next iteration the augmented advantage on action 1 in state 1 is $A_r - 0.45A_c < 0$ (positive cost, zero reward), driving its probability down. After a handful of iterations the policy plays action 1 only in state 2 (where it pays a reward of 1 that justifies the cost), and the cost falls to the budget.

[1] This toy example is just to verify the multiplier-update mechanics: a budget of 0.5 means at most 50% of trajectories should pick the costly action.

3.3 CPO pseudocode

Algorithm: CPO (one update)

Inputs: π_{θ_k}, trust-region δ, budget d.

1. Rollout, estimate A_r, A_c, current violation $c = J_c - d$.
2. Build $g = \widehat{\nabla} J_r$, $b = \widehat{\nabla} J_c$ via score function with advantages.
3. Compute $H^{-1}g$ and $H^{-1}b$ by conjugate gradients on the Fisher–vector product $v \mapsto \widehat{\nabla^2}_\theta \mathrm{KL} \cdot v$.
4. Solve the dual of eq. (2.5); if feasible take the trust-region step, else recovery step.
5. Backtracking line search: shrink the step until both the KL constraint and an estimate of the cost constraint are satisfied.

3.4 FOCOPS pseudocode

Algorithm: FOCOPS (one update)

1. Rollout, estimate A_r, A_c.
2. Solve two 1-D root finds for λ, ν such that the KL and cost constraints hold in expectation.
3. Form the target $\pi^\star$ via eq. (2.6) (only at sampled states).
4. Project onto π_θ by SGD on $\mathrm{KL}(\pi^\star \| \pi_\theta)$.

3.5 P3O pseudocode

Algorithm: P3O (one update)

1. Rollout, compute the PPO surrogate L_r and the cost estimate $\hat{J}_c$.
2. Compute the hinge $H = \max(0, \hat{J}_c - d)$.
3. Backprop $L_r - \kappa H$. The cost gradient flows through $\hat{J}_c$ exactly as it would in any score-function estimator.

Common pitfall

Cost-scale mismatch. If c takes values in $[0, 100]$ and r in $[-1, +1]$, even a tiny multiplier $\lambda = 0.01$ overwhelms the reward gradient. Always normalise the cost stream (e.g. divide by its running mean or by an oracle scale) before plugging it into A_c. Many tuning headaches trace back to this.

4 Implementation walkthrough

We implement PPO-Lagrangian on the `LavaGridWorld` environment in `project/env.py`. The architecture is intentionally minimal: 2 MLPs (one trunk for policy, one for the two value heads) of 64-64 hidden units, one-hot state observations on a 7×7 grid.

4.1 Environment: a constrained lava-grid

```python
@dataclass
class LavaGridWorld:
    rows: int = 7; cols: int = 7
    start = (0, 0); goal = (6, 6)
    lava = ((1,2),(1,3),(2,2),(2,3),(4,3),(4,4),(5,3),(5,4))
    max_steps: int = 80
    slip_prob: float = 0.05
    step_reward: float = -0.01
    goal_reward: float = 1.0
    lava_cost: float = 1.0
    # step(action) -> (next_idx, reward, cost, done, info)
```

Cost is +1 *per visit* to a lava cell — the agent can recover (it does not terminate). This is the standard CMDP setup; the cost is just another scalar channel.

4.2 Networks

Two separate trunks for actor and critic, mostly to keep gradient-graph hygiene clean across PyTorch's autograd. The actor outputs a categorical over 4 actions; the critic has *two heads*, V_r and V_c, with weights tied only through their shared trunk.

```python
class ActorCritic(nn.Module):
    def forward(self, obs):
        h_pi = self.pi_trunk(obs); h_v = self.v_trunk(obs)
        return self.pi_head(h_pi), self.v_head(h_v).squeeze(-1),
        ↪  self.vc_head(h_v).squeeze(-1)
```

4.3 GAE on two streams

```python
def compute_gae(rews, vals, dones, gamma, lam, last_val=0.0):
    T = len(rews); adv = np.zeros(T, dtype=np.float32)
    gae = 0.0; next_val = last_val
    for t in reversed(range(T)):
        nonterm = 1.0 - float(dones[t])
        delta = rews[t] + gamma*next_val*nonterm - vals[t]
        gae = delta + gamma*lam*nonterm*gae
```

```
        adv[t] = gae; next_val = vals[t]
    return adv, adv + np.asarray(vals, dtype=np.float32)
```

The same routine is called twice per update: once with (r_t, V_r, done) and once with (c_t, V_c, done). Cost advantages are *not* normalised — λ has to see the cost scale.

4.4 The combined PPO loss

```
adv_total = (adv_r - self.lam * adv_c) / (1.0 + self.lam)
ratio = torch.exp(logp - old_logps)
clip_adv = torch.clamp(ratio, 1 - eps, 1 + eps) * adv_total
pi_loss = -(torch.min(ratio * adv_total, clip_adv)).mean()
```

The $(1 + \lambda)$ denominator is the variance-reduction trick from Stooke, Achiam, Abbeel 2020: as λ grows, the combined advantage grows linearly, and gradients blow up; dividing by $1 + \lambda$ normalises them.

4.5 Dual update

```
mean_cost = float(np.mean(ep_costs))
self.lam = float(np.clip(self.lam + self.cfg.lam_lr * (mean_cost -
↪  self.cfg.cost_limit),
                         0.0, self.cfg.lam_max))
```

This is the simplest possible projected-ascent rule. PID-Lagrangian (exercise 8) replaces it with a PID controller; FOCOPS replaces it with a two-multiplier root-find. The interface (λ as a Python float on the agent) stays the same.

Common pitfall

Episode boundaries in cost estimates. $\hat{J}_c$ must be the *discounted* cost over completed episodes, not the per-step instant cost. Using per-step costs collapses the budget interpretation. The `train.py` loop accumulates $\sum_t \gamma^t c_t$ explicitly and resets at every done.

5 Empirical results

5.1 Setup

We train PPO-Lagrangian for 80 updates of 512 steps each on the lava grid ($\gamma = 0.99$, GAE $\lambda_{\text{GAE}} = 0.95$, clip 0.2, 4 PPO epochs, minibatch 128, $\eta_\lambda = 0.05$, budget $d = 1.5$). All runs use seed 0 and reproduce in $\sim$30 s on a single CPU core.

5.2 Results

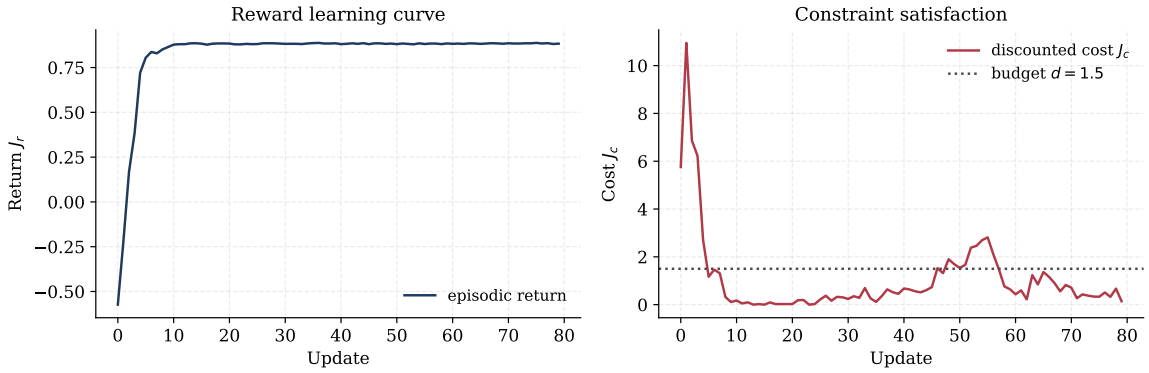


Figure 5.1: Left: episodic reward J_r. Right: discounted cost J_c tracked against the budget $d = 1.5$ (dotted). The policy starts above budget, the multiplier spikes, and the agent learns a route that hits the goal while skirting lava.

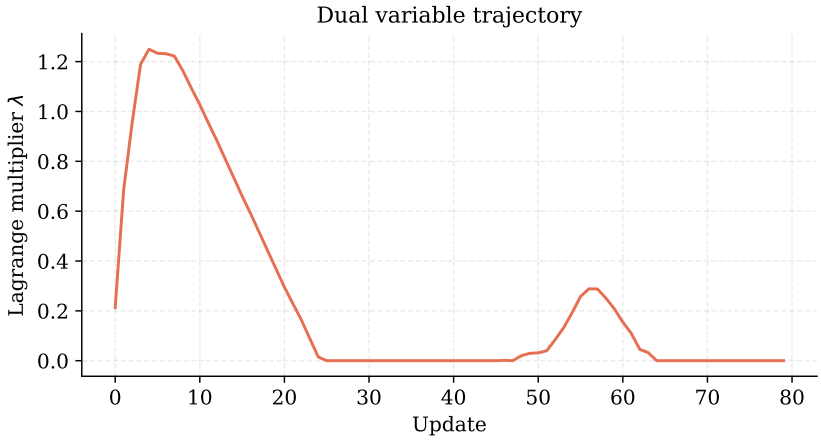


Figure 5.2: Lagrange multiplier λ over training. The sharp early spike is the system pricing the constraint violation; it then settles near 0 once the agent's cost falls below budget (complementary slackness in action).

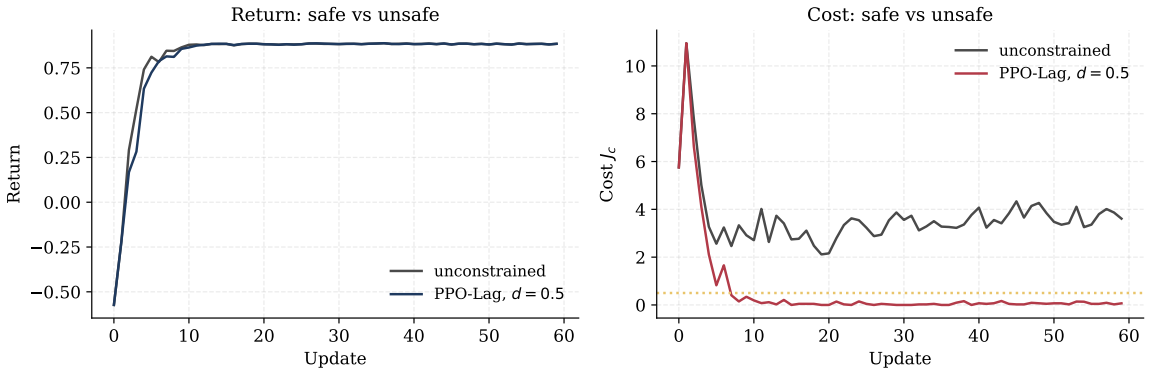


Figure 5.3: Strict-budget PPO-Lag ($d = 0.5$) vs. unconstrained PPO (effectively $d = \infty$). The unconstrained policy converges to the shortest path, which goes *through* lava and incurs high cost; the constrained policy detours, with a small return penalty.

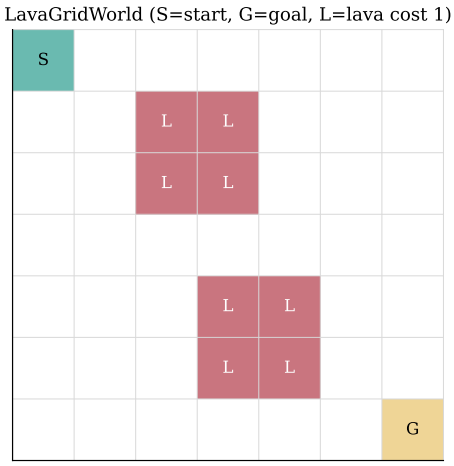


Figure 5.4: The lava gridworld used throughout this chapter.

5.3 Reading the curves

The cost curve clearly shows the dual mechanism: initial overshoot, multiplier-driven correction, eventual convergence to a slack value (below budget). Compare the unconstrained baseline in fig. 5.3, which never sees the constraint signal and stays high on cost forever — just as the theory predicts.

Safety angle

The CMDP is the smallest formalism in which "safety" is a first-class object rather than a hyperparameter. But the formalism is not magical:

- **Constraint specification gaming.** The agent only respects the constraint you wrote, not the one you meant. If c is "visits to a lava cell" and the lava also melts the agent, the agent may find a policy that minimises lava-visits while never actually completing the task — effectively gaming the constraint. Recommended defence: *multiple* cost channels, each capturing one slice of the intended safety property (rate, severity, duration).
- **Asymptotic only.** The Lagrangian guarantees feasibility at the saddle point. During training there is no guarantee. For real deployment you must wrap the policy in a

runtime safety filter (*week 23*: shielding, CBFs).

- **Multiplier oscillation under cost noise.** The dual update is a noisy projected descent on L; when $\hat{J}_c$ has high variance, λ can oscillate wildly, driving the policy back and forth between greedy and risk-averse modes. PID-Lagrangian (Stooke et al. 2020, arXiv:2007.03964) damps these oscillations.

References: Ray, Achiam, Amodei 2019, arXiv:1910.01708, "Benchmarking safe exploration in deep RL"; Stooke, Achiam, Abbeel 2020, arXiv:2007.03964, "Responsive safety in RL by PID Lagrangian methods".

Matching application

For the synthetic ATS candidate-matching application, the CMDP form is the right level of abstraction for fairness. Define:

- $r(s, a)$ = match-quality score (skill overlap, recruiter-rating proxy) of presenting candidate a in context s.
- $c(s, a) = \left| \mathbb{1}[g(a) = \text{minority}] - \rho \right|$ where ρ is the target group share and $g(a)$ the candidate's protected attribute. A discounted sum of c measures the policy's average departure from demographic parity.
- d is the maximum tolerated parity gap (set by counsel / DEI policy).

PPO-Lagrangian then learns a re-ranking policy that maximises match quality *subject to* the parity gap not exceeding d. The dual variable λ is the marginal cost of one extra unit of parity violation — literally what a regulator should be auditing. We will productionise this exact setup in the week-24 capstone.
For synthetic insurance lead routing the analogue is: r = expected gross-written-premium contribution; c = saturation of any one broker beyond their capacity quota; d = broker-quota slack policy. PPO-Lag respects the broker fairness constraint without manual scalarisation.

6 Exercises

See `workbook.ipynb` for graded exercises with hints and an auto-graded reference. A summary:

- **Ex. 1 ($\star$):** one-step Lagrangian dual update.
- **Ex. 2 ($\star$):** P3O hinge subgradient.
- **Ex. 3 ($\star\star$):** GAE for the cost stream.
- **Ex. 4 ($\star\star$):** FOCOPS closed-form non-parametric optimal.
- **Ex. 5 ($\star\star$):** empirical CVaR.
- **Ex. 6 ($\star\star$):** train PPO-Lag, verify constraint satisfaction.
- **Ex. 7 ($\star\star\star$):** compare constrained vs. unconstrained; explain the gap.
- **Ex. 8 ($\star\star\star$):** PID-Lagrangian one-step update.

7 Further reading

The bibliography for this week is in `extended_reading.pdf`:

- Altman, *Constrained Markov Decision Processes*, CRC 1999 — the canonical reference.
- Ray, Achiam, Amodei, "Benchmarking safe exploration in deep RL", arXiv:1910.01708 (2019).
- Achiam, Held, Tamar, Abbeel, "Constrained Policy Optimization", arXiv:1705.10528 (2017).
- Zhang, Vuong, Ross, "FOCOPS", arXiv:2002.06506 (2020).
- Stooke, Achiam, Abbeel, "Responsive safety by PID Lagrangian", arXiv:2007.03964 (2020).
- Zhang, Vuong, Ross, "P3O", arXiv:2205.11814 (2022).

Sourced from `research/safety_rl_sota.md`, sections 1 and 3.